

```

module alu_4bit(
    input [3:0] A,
    input [3:0] B,
    input [2:0] Sel,
    output reg [3:0] Result,
    output reg Carry
);

always @(*) begin
    Carry = 0;

    case(Sel)
        3'b000: {Carry, Result} = A + B; // Addition
        3'b001: {Carry, Result} = A - B; // Subtraction
        3'b010: Result = A & B;          // AND
        3'b011: Result = A | B;          // OR
        3'b100: Result = A ^ B;          // XOR
        3'b101: Result = ~(A | B);       // NOR
        3'b110: Result = A << 1;         // Left Shift
        3'b111: Result = A >> 1;         // Right Shift
        default: Result = 4'b0000;
    endcase
end

endmodule

```

```
`timescale 1ns/1ps
```

```
module alu_4bit_tb;
```

```
    reg [3:0] A, B;
```

```
    reg [2:0] Sel;
```

```
    wire [3:0] Result;
```

```
    wire Carry;
```

```
    alu_4bit uut (
```

```
        .A(A),
```

```
        .B(B),
```

```
        .Sel(Sel),
```

```
        .Result(Result),
```

```
        .Carry(Carry)
```

```
    );
```

```
    initial begin
```

```
        A = 4'b1010; // 10
```

```
        B = 4'b0011; // 3
```

```
        Sel = 3'b000; #10; // Add
```

```
        Sel = 3'b001; #10; // Subtract
```

```
        Sel = 3'b010; #10; // AND
```

```
        Sel = 3'b011; #10; // OR
```

```
        Sel = 3'b100; #10; // XOR
```

```
        Sel = 3'b101; #10; // NOR
```

```
        Sel = 3'b110; #10; // Left Shift
```

```
        Sel = 3'b111; #10; // Right Shift
```

```
    $finish;
```

```
end
```

```
endmodule
```